Lab 4: Functional Dependencies

Reda Lakhlouf

October 2026

The FDs are given in two parts. Part 1 contains the FDs that come directly from the key of each relation. Part 2 contains the FDs that come from the foreign keys.

Each FD is written inside one relation as `Relation.attribute`. The code in brackets is the requirement of Lab 1 that gives the FD. A key determines all the other attributes of its relation, and we group the right sides with the union rule: if $X \to Y$ and $X \to Z$ then $X \to YZ$. A relation that contains only its key has no other FD, so we write "key only".

1 Part 1: FDs from the keys

1.1 People

- `Person.pid` → `firstName, lastName, email, affiliation, startDate, endDate` (PE1: a person has an internal identifier)
- `Student.pid` → `program` (PE3: a student is enrolled in one program)
- `Employee.pid` → `phone, office, supervisorPid` (PE4: an employee has a phone and an office. PE5: an employee reports to one supervisor, and the dean has none, so `supervisorPid` can be `NULL`)
- `Academic(pid)` and `NonAcademic(pid)`: key only
- `Faculty.pid` → `position` (PE4: faculty has a position)
- `Administrative.pid` → `position` (PE4)
- `Technical.pid` → `position` (PE4)
- `Advises(academicPid, studentPid)`: key only (PE6). A student can have several advisors and an academic can advise several students, so neither attribute determines the other.

Some rules look like FDs but are not. `supervisorPid` does not determine `pid`, because a supervisor has several subordinates. "An employee cannot report to himself" and "faculty and administrative staff report to the dean" limit the possible values, so they are constraints. "Each student has at least one advisor" is a cardinality constraint.

1.2 Laboratories, Projects, and Budgets

- `Laboratory.labId` → `name, building, roomNumber, discipline, supervisorPid` (LA1: a laboratory has these attributes. LA4: it has exactly one faculty supervisor)

- `Attached(pid, labId)`: key only (LA3)
- `ResearchProject.code` → `title, startDate, endDate, status` (PR1)
- `Participates.(pid, projectCode)` → `role` (PR4: a person has exactly one role in a project)
- `Budget.budgetLine` → `amountGranted, amountDisbursed, startDate, endDate, managerPid` (BD1: the budget attributes. BD2: exactly one academic manages it)
- `FundsPrj(budgetLine, projectCode)` and `FundsLab(budgetLine, labId)`: key only (BD3)

Checks and exceptions:

- For `Participates`, `pid` alone does not determine `role`, because a person can be in several projects (PR2) with different roles. `projectCode` alone does not either, because a project has a PI, a Co-PI and collaborators (PR3). The closure of each one is only itself, so we need the whole key.
- "Each project has exactly one PI" would be (`projectCode`, `role` = PI) → `pid`. This is true only for one value of `role`, so it is a constraint and not an FD.
- `supervisorPid` does not determine `labId`, because a faculty member supervises several laboratories (LA4). In the same way `managerPid` does not determine `budgetLine` (BD2).

1.3 Equipment and Certifications

- `EquipmentModel.modelId` → `commercialName, manufacturer, category, requiredEnvironment, trainingMandatory` (EQ2)
- `EquipmentUnit.serialNo` → `acquisitionDate, purchaseCost, status, portable, modelId, labId` (EQ3: the unit attributes. EQ5: a unit has exactly one model. EQ4: only the current laboratory is kept, so there is one `labId`)
- `Certification.code` → `title, issuingAuthority, validityPeriod, safetyLevel` (CE2)
- `Requires(modelId, certCode)`: key only (CE6)
- `Holds.(pid, certCode)` → `issueDate, expirationDate, grade` (CE4: the attributes of an obtained certification. CE5: only the latest award is kept, so there is one tuple per pair)

Checks and exceptions:

- `modelId` does not determine `serialNo` (EQ5), and `labId` does not determine `serialNo` (LA2), because a model has many units and a laboratory hosts many units.
- If the model attributes were stored in `EquipmentUnit`, we would have `modelId` → model attributes, and `modelId` is not a key there. It is the same problem as $R \to W$ in the `Hourly_Emps` example of the course: update, insertion and deletion anomalies. EQ1 (model and unit are separate) is the decomposition that removes it.
- In `Holds`, `(pid, certCode)` → `pid` is true by reflexivity, but it is trivial so we do not list it. `pid` alone does not determine `certCode`, because a person can hold several certifications (CE3).

1.4 Reservations

- `Reservation.resId` → `submissionTS, plannedStart, plannedEnd, purpose, status, madeByPid, projectCode, approverPid` (RE2: the reservation attributes. RE1: it is made by exactly one person for one project. RE3: it has at most one approver, so `approverPid` can be `NULL`)
- `Reserves(resId, serialNo)`: key only (RE1: at least one unit per reservation)

Checks and exceptions:

- `madeByPid` does not determine `projectCode`, because a person makes several reservations. `approverPid` does not determine `resId` (RE3).
- "Only certified people can reserve" (CE7), "one user at a time" (EQ7), "reserve before use" (RE4) and "administrative staff cannot reserve" (RE1) depend on other tables or on time. They are constraints and not FDs.

1.5 Maintenance and Calibration

- `Maintenance.(serialNo, startTS)` → `endTS, type, description, cost, outcome, technicalPid` (MA2: a maintenance is identified by the serial number and the start timestamp. MA3: it is done by one technical staff member)
- `CalibrationRecord.(serialNo, calibDate)` → `calibrationType, result, nextDueDate, remarks` (CA2: the attributes. CA3 and CA4: it belongs to one unit and is identified by its date)

Checks and exceptions:

- The closure of `serialNo` alone is only `serialNo`, because a unit has several maintenances (MA1). The closure of `startTS` alone is only `startTS`, because two units can be in maintenance at the same time. So the pair is the key. It is the same for `CalibrationRecord` with `calibDate`.
- `CalibrationRecord` has no `technicalPid`, because the person who calibrates is not recorded (CA1).

1.6 Consumables, Suppliers, and Stock

- `Consumable.consId` → `name, unitOfMeasure, hazardLevel, reorderThreshold` (CO2: one threshold per consumable, the same in every laboratory)
- `Supplier.suppId` → `name, contactEmail, phone` (CO3)
- `Supplies.(suppId, consId, labId)` → `unitPrice` (CO4: the price depends on the supplier, the consumable and the laboratory together)
- `Stocks.(labId, consId)` → `quantityOnHand, lastRestockDate, storageCondition, technicalPid, monitoringSince` (CO6 and CO7: quantity and dates for each laboratory and consumable. CO8: exactly one technical staff member monitors this stock, since `monitoringSince`)
- `Consumes.(resId, labId, consId)` → `quantityUsed` (CO9: quantity used by a reservation)

Checks and exceptions:

- For `Supplies`, the closure of any pair of the three attributes is only the pair itself. In particular `(suppId, consId)` does not determine `unitPrice`, because the same supplier gives different prices to different laboratories. So the three attributes are needed.
- For `Stocks`, `labId` alone and `consId` alone determine nothing else, because the same consumable is in several laboratories with different quantities (CO6). `technicalPid` does not determine `monitoringSince`, because the date belongs to one laboratory and consumable pair.
- `Supplies` and `Stocks` are separate because a supply arrangement can exist without stock (CO5).
- `resId` does not determine `consId` in `Consumes`, because a reservation can consume several consumables (CO9).

In every relation, the left side of each FD above is a key of the relation. So there is no FD like $R \to W$ with a left side that is not a key.

2 Part 2: FDs from the foreign keys

We use two rules. By reflexivity, a key determines the foreign key inside it. By transitivity, if $A \to B$ and $B \to C$ then $A \to C$. So when a foreign key points to the key of another relation, it determines all the other attributes of that relation, and so does the key of the relation that contains the foreign key. Attributes with the same name in several relations (`name`, `phone`, `startDate`, `endDate`, `status`) are written with the name of their relation.

2.1 People

- `Student.pid`, `Employee.pid` → `Person.firstName, Person.lastName, Person.email, Person.affilia` `Person.startDate, Person.endDate` (a student and an employee are persons, so their `pid` is a `Person.pid`)
- `Academic.pid`, `NonAcademic.pid`, `Faculty.pid`, `Administrative.pid`, `Technical.pid` → `Employee.phone, Employee.office, Employee.supervisorPid`, and then → `Person.firstName, Person.lastName, Person.email, Person.affiliation, Person.startDate, Person.endDate` (these are employees, so the chain goes through `Employee.pid`)
- `Employee.supervisorPid` → `phone, office, supervisorPid, firstName, lastName, email, affiliation, startDate, endDate` of the supervisor, because it points to `Employee.pid`
- `Advises.academicPid` → `Employee.phone, Employee.office, Employee.supervisorPid, Person.firstName, Person.lastName, Person.email, Person.affiliation, Person.startDate, Person.endDate`
- `Advises.studentPid` → `Student.program, Person.firstName, Person.lastName, Person.email, Person.affiliation, Person.startDate, Person.endDate`

2.2 Laboratories, Projects, and Budgets

- Laboratory.supervisorPid → Faculty.position, Employee.phone, Employee.office, Employee.supervisorPid, Person.firstName, Person.lastName, Person.email, Person.affiliatio Person.startDate, Person.endDate. Since labId determines supervisorPid, a laboratory determines all the attributes of its supervisor.
- Attached.pid → Person.firstName, Person.lastName, Person.email, Person.affiliation, Person.startDate, Person.endDate
- Attached.labId → Laboratory.name, Laboratory.building, Laboratory.roomNumber, Laboratory.discipline, Laboratory.supervisorPid
- Participates.pid → Person.firstName, Person.lastName, Person.email, Person.affiliation, Person.startDate, Person.endDate
- Participates.projectCode → ResearchProject.title, ResearchProject.startDate, ResearchProje ResearchProject.status
- Budget.managerPid → Employee.phone, Employee.office, Employee.supervisorPid, Person.firstN Person.lastName, Person.email, Person.affiliation, Person.startDate, Person.endDate. Since budgetLine determines managerPid, a budget determines all the attributes of its manager.
- FundsPrj.budgetLine → Budget.amountGranted, Budget.amountDisbursed, Budget.startDate, Budget.endDate, Budget.managerPid
- FundsPrj.projectCode → ResearchProject.title, ResearchProject.startDate, ResearchProject.e ResearchProject.status
- FundsLab.budgetLine → Budget.amountGranted, Budget.amountDisbursed, Budget.startDate, Budget.endDate, Budget.managerPid
- FundsLab.labId → Laboratory.name, Laboratory.building, Laboratory.roomNumber, Laboratory.discipline, Laboratory.supervisorPid

2.3 Equipment and Certifications

- EquipmentUnit.modelId → EquipmentModel.commercialName, EquipmentModel.manufacturer, EquipmentModel.category, EquipmentModel.requiredEnvironment, EquipmentModel.trainingMandat Since serialNo determines modelId, a unit determines all the attributes of its model.
- EquipmentUnit.labId → Laboratory.name, Laboratory.building, Laboratory.roomNumber, Laboratory.discipline, Laboratory.supervisorPid
- Requires.modelId → EquipmentModel.commercialName, EquipmentModel.manufacturer, EquipmentModel.category, EquipmentModel.requiredEnvironment, EquipmentModel.trainingMandat
- Requires.certCode → Certification.title, Certification.issuingAuthority, Certification.val Certification.safetyLevel
- Holds.pid → Person.firstName, Person.lastName, Person.email, Person.affiliation, Person.startDate, Person.endDate

- Holds.certCode → Certification.title, Certification.issuingAuthority, Certification.validi
  Certification.safetyLevel

2.4 Reservations

- Reservation.madeByPid → Person.firstName, Person.lastName, Person.email, Person.affiliatio
  Person.startDate, Person.endDate (the person who made the reservation)
- Reservation.approverPid → the same six attributes of Person (the approver)
- Reservation.projectCode → ResearchProject.title, ResearchProject.startDate, ResearchProjec
  ResearchProject.status
- Reserves.resId → Reservation.submissionTS, Reservation.plannedStart, Reservation.plannedEn
  Reservation.purpose, Reservation.status, Reservation.madeByPid, Reservation.projectCode,
  Reservation.approverPid
- Reserves.serialNo → EquipmentUnit.acquisitionDate, EquipmentUnit.purchaseCost,
  EquipmentUnit.status, EquipmentUnit.portable, EquipmentUnit.modelId, EquipmentUnit.labId

2.5 Maintenance and Calibration

- Maintenance.serialNo → EquipmentUnit.acquisitionDate, EquipmentUnit.purchaseCost,
  EquipmentUnit.status, EquipmentUnit.portable, EquipmentUnit.modelId, EquipmentUnit.labId
- Maintenance.technicalPid → Technical.position, Employee.phone, Employee.office,
  Employee.supervisorPid, Person.firstName, Person.lastName, Person.email, Person.affiliatio
  Person.startDate, Person.endDate
- CalibrationRecord.serialNo → EquipmentUnit.acquisitionDate, EquipmentUnit.purchaseCost,
  EquipmentUnit.status, EquipmentUnit.portable, EquipmentUnit.modelId, EquipmentUnit.labId

2.6 Consumables, Suppliers, and Stock

- Supplies.suppId → Supplier.name, Supplier.contactEmail, Supplier.phone
- Supplies.consId → Consumable.name, Consumable.unitOfMeasure, Consumable.hazardLevel,
  Consumable.reorderThreshold
- Supplies.labId → Laboratory.name, Laboratory.building, Laboratory.roomNumber,
  Laboratory.discipline, Laboratory.supervisorPid
- Stocks.labId → Laboratory.name, Laboratory.building, Laboratory.roomNumber, Laboratory.dis
  Laboratory.supervisorPid
- Stocks.consId → Consumable.name, Consumable.unitOfMeasure, Consumable.hazardLevel,
  Consumable.reorderThreshold
- Stocks.technicalPid → Technical.position, Employee.phone, Employee.office, Employee.superv
  Person.firstName, Person.lastName, Person.email, Person.affiliation, Person.startDate,
  Person.endDate

- `Consumes.resId` → `Reservation.submissionTS, Reservation.plannedStart, Reservation.plannedEn Reservation.purpose, Reservation.status, Reservation.madeByPid, Reservation.projectCode, Reservation.approverPid`
- `Consumes.(labId, consId)` → `Stocks.quantityOnHand, Stocks.lastRestockDate, Stocks.storageCo Stocks.technicalPid, Stocks.monitoringSince` (the foreign key points to `Stocks`)

3 Hypotheses

The requirements do not give the following FDs. They are true only if we make the assumption.

- If an email belongs to only one person, then `Person.email` → `pid`. So `email` is also a candidate key of `Person`.
- If a supplier's contact email is unique, then `Supplier.contactEmail` → `suppId`.
- If a phone number is never shared, then `Employee.phone` → `pid`.
- If an office is never shared, then `Employee.office` → `pid`.
- If a room hosts at most one laboratory, then `Laboratory.(building, roomNumber)` → `labId`. So `(building, roomNumber)` is also a candidate key.
- If a project title is unique, then `ResearchProject.title` → `code`.
- If the expiration date is always the issue date plus the validity period, then `Holds.(certCode, issueDate)` → `expirationDate`. Here `(certCode, issueDate)` is not a key of `Holds`, so the expiration date would be repeated for each person who got the certification on the same day.

The first six hypotheses give a new candidate key, so the left side is a key. Only the last one gives a left side that is not a key.

4 Report

4.1 Role of each member

I did all the tasks myself.

1. I reviewed Lecture 4, which we had already seen in class, to be sure I master what I needed for this lab: the definition of an FD, how it relates to keys, Armstrong's axioms and attribute closure.
2. I went back to Lab 1 and Lab 2 to understand how the relations are linked through primary and foreign keys.
3. I wrote the FDs, first those from the keys (Part 1) and then those from the foreign keys (Part 2).

4.2 Key challenges

- The schema has thirty relations, so there were many dependencies. I split the work in two parts: the FDs from the keys, and the FDs from the foreign keys. In each part I followed the sections of Lab 2, and I went back to Lab 1 each time to check the cardinalities.
- My first draft put attributes of several relations in the same FD, for example the attributes of `Person` in an FD of `Student`. I corrected it. The FDs of Part 1 stay inside one relation, and the FDs that go from one relation to another are in Part 2, with the name of the relation in front of each attribute.
- Some rules look like FDs but are not, for example "exactly one PI" or "at least one advisor". I kept them out of the lists and wrote them in the relation they concern.

4.3 Justification of the choices

- For each relation I started with the key, because a key determines all the other attributes of its relation.
- In Lab 2, a relationship with "at most one" or "exactly one" on one side became a foreign key inside a table (Supervises, Manages, InstanceOf, LocatedIn, MadeBy, For, Approves, DoneBy, Monitors, ReportsTo). Each key value has one value for this foreign key, so it gives an FD. For example, `labId` $\rightarrow$ `supervisorPid`. When the relationship is "at most one", the foreign key can be `NULL`.
- M:N relationships without attributes have all their attributes in the key, so they are "key only".
- For the composite keys (`Supplies`, `Stocks`, `Maintenance`, `CalibrationRecord`, `Participates`), I computed the closure of each part of the key. None of them determines the other attributes, so no attribute depends on only a part of the key.
- The FDs from the foreign keys follow from transitivity, so I listed them for every foreign key in Part 2.
- The split between model and unit in Lab 1 is a decomposition. If the model attributes were in the unit table, `modelId` would determine them without being a key, like $R \rightarrow W$ in the course example.
- Every FD comes from a requirement and not from sample data, because an instance can show that an FD is false but cannot show that it is true.

4.4 Assumptions

I only added FDs that need an assumption in the Hypotheses section, and each one is written as "if we assume this, we get this FD". Without them, the FDs of Part 1 and Part 2 are the only ones.